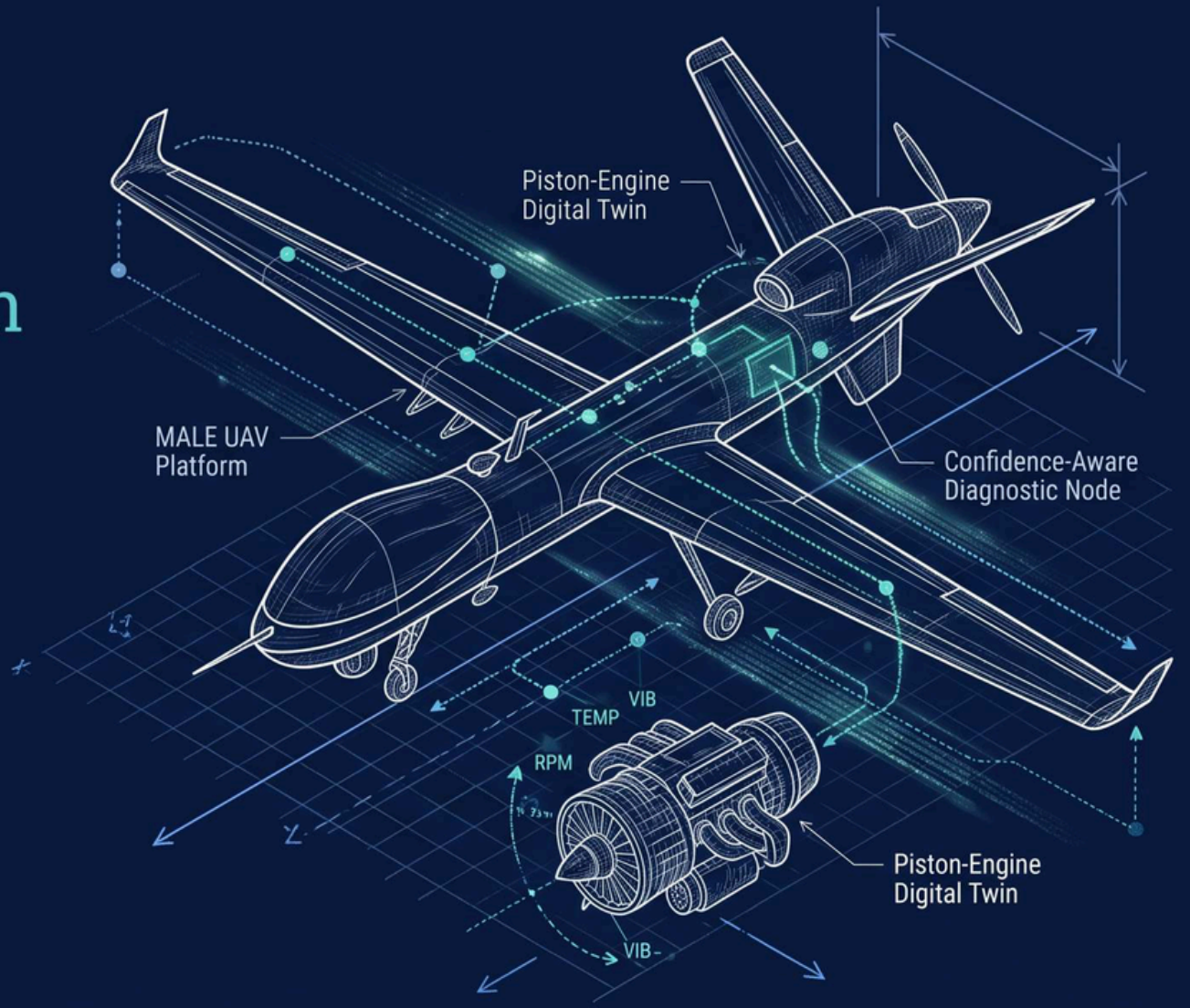
AeroTwin

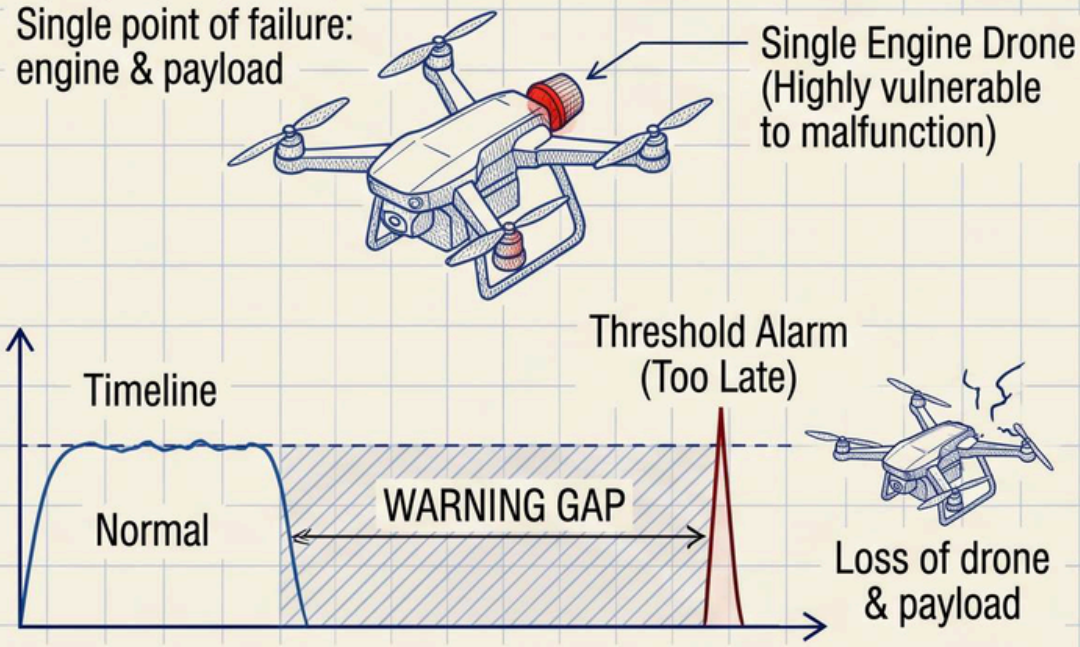
AI-Powered Digital Twin for Predictive UAV Engine Health

- Project: AeroTwin — AI-Powered Digital Twin for Predictive UAV Engine Health
- Primary platform: MALE UAV (Medium Altitude Long Endurance) / piston-engine monitoring concept
- Prototype: Healthy-Data Learning & Confidence-Aware Demo
- Classification: Project concept + proposed implementation context (not a validated production system)



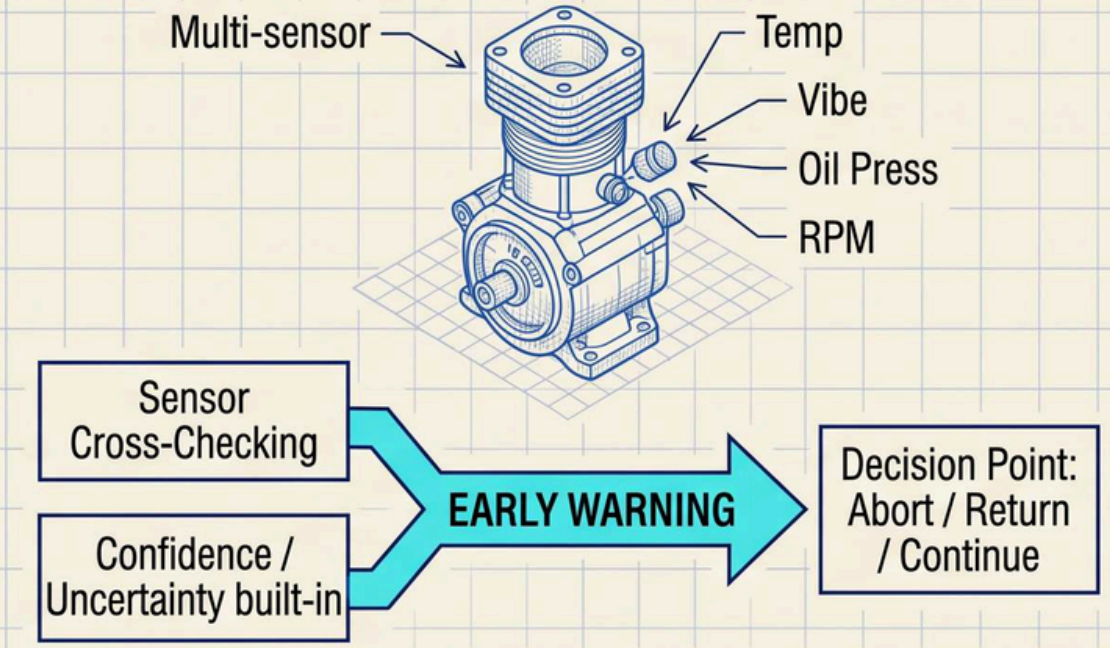
Problem Statement: Why It Matters

REACTIVE MONITORING



- Abnormal behavior develops before threshold alarms activate
- Reactive monitoring identifies problems only after a limit is crossed
- Sudden or late-detected anomalies (overheating, vibration) lead to failure

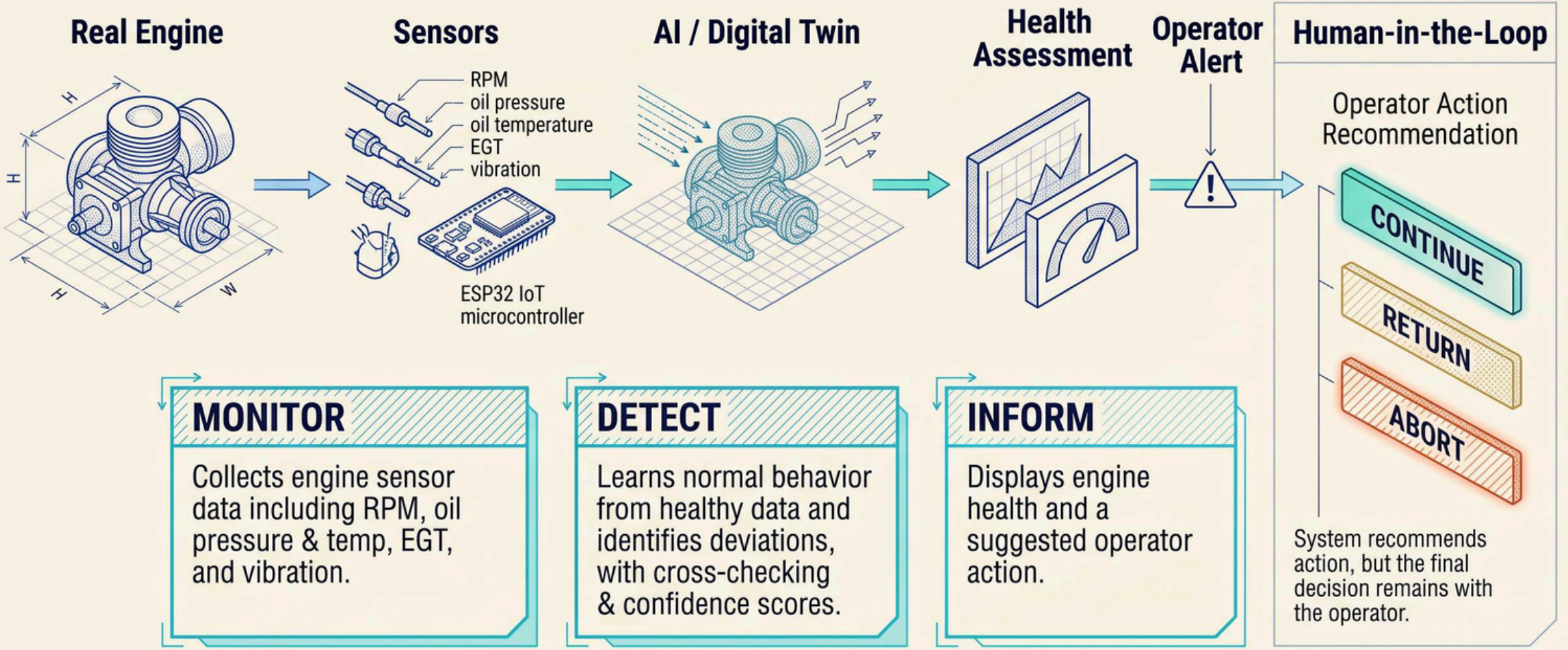
EARLY ANOMALY DETECTION (PROACTIVE)



- Proactive detection catches early trends before complete failure
- Built-in confidence cross-checking avoids critical false alarms
- Empowers operators with early, understandable warnings to save the mission

AeroTwin: Proposed Solution

An onboard engine-health monitoring system for drone piston engines.



MONITOR

Collects engine sensor data including RPM, oil pressure & temp, EGT, and vibration.

DETECT

Learns normal behavior from healthy data and identifies deviations, with cross-checking & confidence scores.

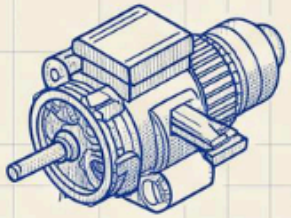
INFORM

Displays engine health and a suggested operator action.

End-to-End Workflow (16 Steps)

SENSING

1. Real Drone Engine



2. Engine Sensors

RPM
EGT
oil pressure/temp
vibration
EGT

3. Live Sensor Data

digital
data 2
digital
data 3

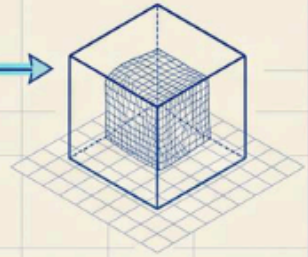
4. Onboard Monitoring Computer

AI PROCESSING

5. Data Pre-processing

Data
Cleaning and scaling
Scaing

6. Healthy-Data AI Model / Digital Twin



7. Prediction

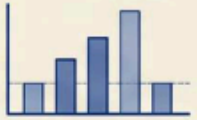
Simulated model sensor pathways

8. Actual vs Predicted Comparison

Live
Expected values

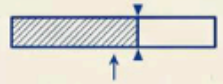
HEALTH ASSESSMENT

9. Novelty / Anomaly Score



Abnormality measure

10. Confidence / Uncertainty



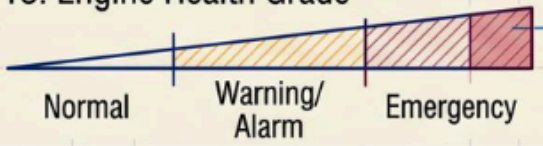
Uncertainty estimate indicator

11. Cross-check / Sensor Health

Sensor
Sensor health

12. Engine Health Assessment

13. Engine Health Grade



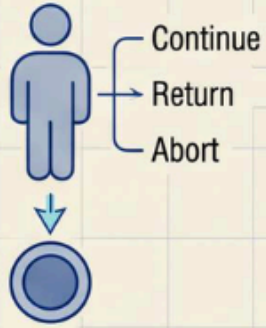
Normal
Warning/ Alarm
Emergency

HUMAN ACTION

14. Operator Dashboard / Alert



15. Human Operator Decision



Continue
Return
Abort

16. Early Warning → Action Before Failure

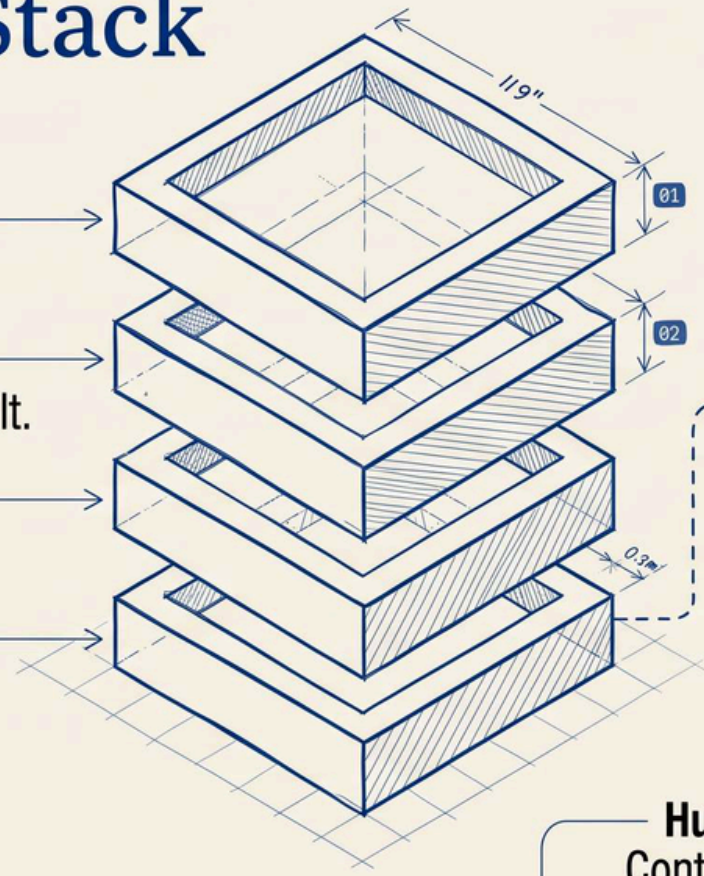
System Architecture & Tech Stack

FRONTEND LAYER (Next.js, React, Tailwind CSS, Recharts/Plotly)
- Live engine-health dashboard; trends, alerts & explanations; operator viz.

ML LAYER (PyTorch Autoencoder, Scikit-learn, NumPy)
- Train on healthy data; detect anomalies; calculate health score; analyze fault.

BACKEND LAYER (FastAPI, WebSocket, Python)
- Expose APIs; handle real-time data; integrate frontend, ML & database.

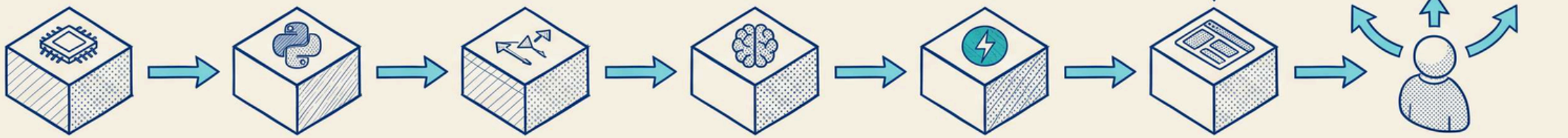
DATA LAYER (Python, NumPy, Pandas, SQLite/CSV)
- Simulate/collect sensor data; store telemetry & history; provide data for ML/viz.



ESP32
(Optional)

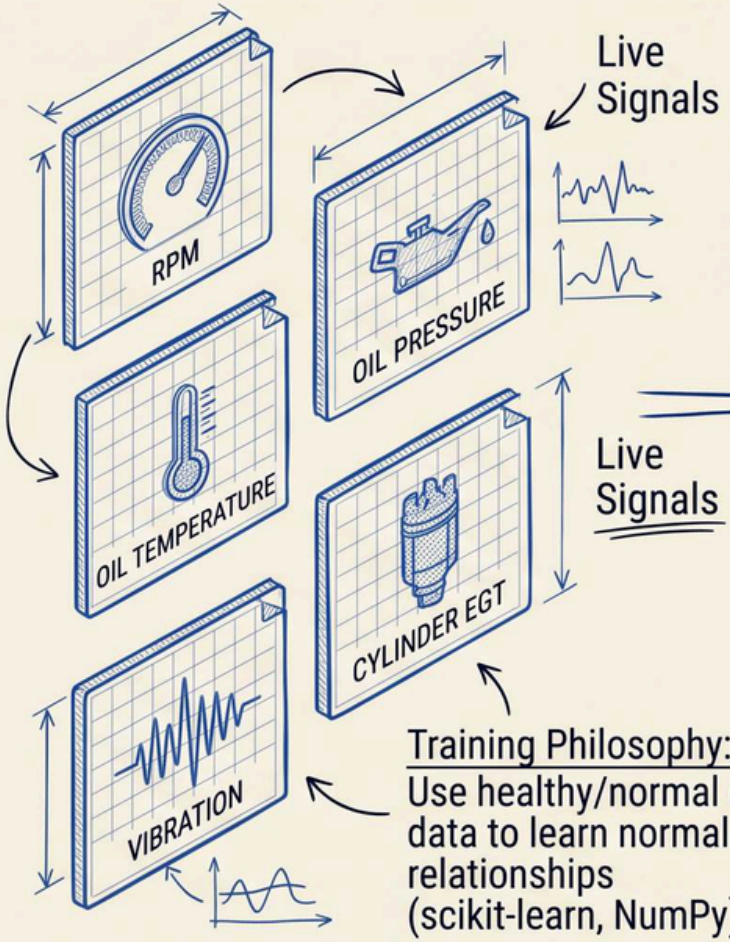
Optional IoT input for physical demo setup

PROPOSED DATA PATH



ML/Digital Twin Concept & Sensor Inputs

PROTOTYPE SENSOR INPUTS



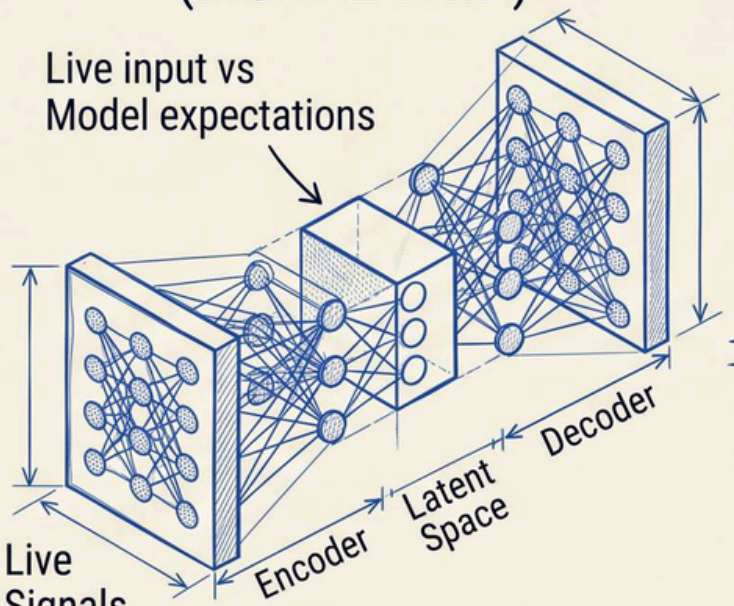
Live Signals

Live Signals

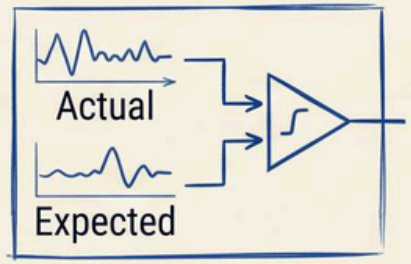
Training Philosophy:
Use healthy/normal engine data to learn normal relationships (scikit-learn, NumPy)

PyTorch AUTOENCODER (DIGITAL TWIN)

Live input vs Model expectations



Live Signals



Deviation → Novelty / Anomaly Signal

HEALTH OUTPUT & DASHBOARD



Low confidence = unfamiliar condition (not automatically a fault)

Cross-check: Other signals reduce false alarms

OPTIMAL MONITORING PHASES

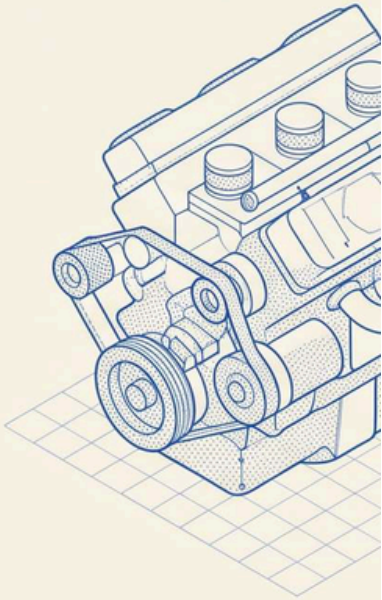
- ☑ Pre-start, Startup, Idle, Pre-flight (Reliable Stationary)
- ☑ Taxi, Takeoff, Climb (Less Suitable)

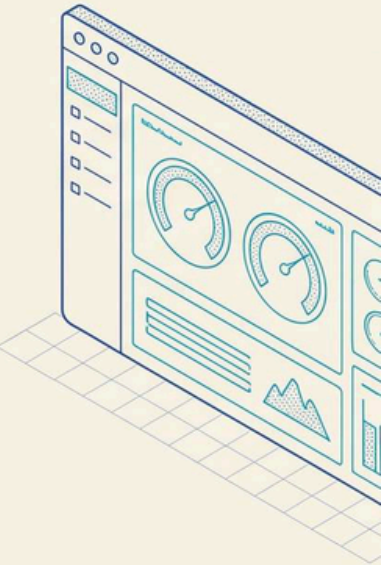
PLOTTYTE 9TROPEIT |

Existing Approaches vs AeroTwin

Comparison table — Approach | Limitation | AeroTwin's Target

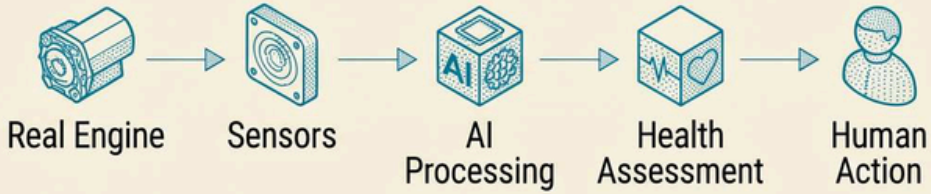
Existing Approach	Limitation	AeroTwin's Target (Proposed)
Cockpit gauges & human observation	Human-dependent; UAV operator is remote; threshold-only checks can miss gradual trends	Dashboard-based, understandable earlier warnings (piston-engine)
Digital engine monitor / ECU-FADEC	Alarms can be late; sensor failures exist; older engines lack ECUs	Early anomaly detection before thresholds; sensor cross-checking
Statistical pattern recognition	Piston-relevant but cited studies lack tested accuracy & are engine-specific	Adopts phase-wise/cross-check ideas for piston-engine monitoring
Digital twin + Transformer	Heavy compute & 3D required; turbofan-focused; simulated validation only	Lightweight, lower-compute framework tailored to piston engines
Distributed digital twin	Gas-turbine focused; detection only, lacking operational decision support	Confidence-aware, cross-checked approach with Continue/Return/Abort suggestions

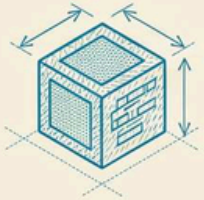




*Note: NASA C-MAPSS is turbofan simulated data, used only as a labeled proxy, not piston-engine validation.

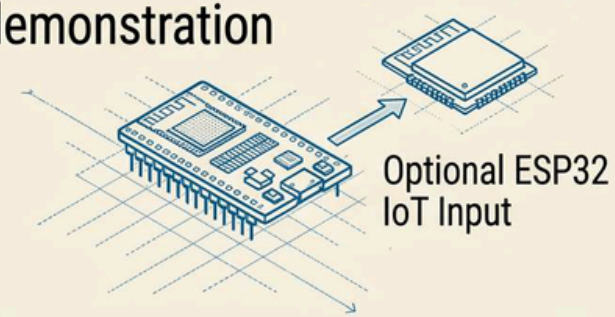
Feasibility, Viability & Impact

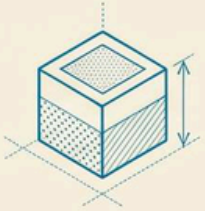




TECHNICAL FEASIBILITY

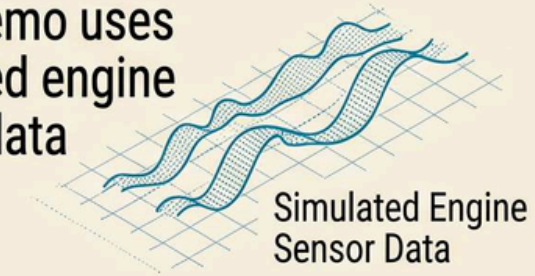
- Python/ML software prototype (backend streaming, web dashboard).
- Lightweight, onboard-oriented processing concept.
- Optional ESP32 input for physical demonstration



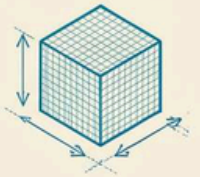


PRACTICAL CONSTRAINTS

- Real piston-engine failure data is difficult to obtain.
- Initial demo uses simulated engine sensor data



- Real piston-engine validation remains necessary.



EXPECTED VALUE
(Intended Benefits)

- Earlier visibility into abnormal engine behavior.
- Potential reduction in avoidable false alarms through cross-checking.
- Clearer health information for operators.
- Potential support for maintenance planning and mission decisions.

Note: No percentage improvements, cost savings, or performance statistics are claimed.

PROTOTYPE

LINK:

GITHUB
DASHBOARD

